

Project 2

I²C bit banging

EELE 465

Acknowledgement This handout is based off of previous handouts by Randy Larimer and Cory Mettler.

Project goal Bit bang the I²C protocol to communicate with the Analog Discovery 2 and a real-time clock (RTC).

This lab will introduce you to “[bit banging](#)” the I²C protocol. Bit banging means that you personally develop the communication code rather than using the MCU’s embedded modules. This lab will require manipulating GPIO pins and using the delay loop techniques we explored earlier.

You will program this project in assembly. By the end of this project, you will be quite familiar with low-level MCU programming and how MCUs work 🧐.

To help you develop your I²C code, this project is broken up into several parts. Future projects will not give you as much guidance. The goal is to give you small bits that you can develop and test; in the future, you’ll be expected to break up the projects yourself.



This project will require a significant amount of effort. I highly suggest you

- Test your code early and often.
- Be systematic.
- Develop modular code.

GitHub Classroom assignment The [GitHub assignment](#) is a group assignment. The first person in your team to [accept the assignment](#) will need to create a team name. Please use both members’ first names, e.g., lily-trevor. The second team member to accept the assignment just needs to select the correct team to join.

Outcomes

After this lab you should be able to:

- Set up the MSP430FR2355 microcontroller for general purpose I/O (GPIO) on a port/pin of your choosing.
- Implement I2C using bit banging to communicate with another device.
- Setup and use the Analog Discovery 2’s logic analyze and/or the Mixed Signal Oscilloscope’s (MSO) bus analysis feature to observe I2C signals coming out of the MSP430 Launchpad Board.
- Use the Analog Discovery 2’s protocol analyzer as an I²C peripheral.
- Interface with an I²C real-time clock.

Background

I²C

I²C is a synchronous, two-wire serial communication protocol. It has two signals: SCL (clock) and SDA (data). An I²C [bus](#) can have multiple controllers and multiple targets/peripherals.

Rather than me doing a mediocre job of explaining I²C, I will refer you to several application (app) notes on I²C. After you graduate, you will commonly learn from app notes and datasheets; so why delay—let's start having fun today! (by reading app notes 🤖)

Here are some app notes and manuals I suggest you start with:

- [TI: A basic guide to I²C](#)
- [The official I²C bus-specification and user manual](#)
- [An old app note with some bit banging assembly code samples](#)

These app notes are also on Canvas under Resources/I2C.



The official bus-specification and user manual is a great and should be one of your primary resources

Analog Discovery

The following tutorials may be useful for learning how to use the protocol analyzer and logic analyzer:

- [waveforms protocol analyzer guide](#)
- [waveforms logic analyzer guide](#)

Oscilloscope bus analyzer

If you wish to use the oscilloscope's bus analyzer function, there is a [one-page document on Canvas](#).

Teamwork

Starting with Project 3, you will need to distribute the project workload approximately evenly among the partners before starting the project. Most projects will have some tasks that are easy to delegate and other tasks that must be developed as a group effort (this means that you are sitting at the same lab bench, working on these steps at the same time as your partner).

Project 2 is an exception. There is not much that can be divided in this particular project—each student needs to understand each step. So, in this project, teams are expected to develop each part of the code with their teammates as a group effort.

In future projects, you will be allowed to divide and conquer the tasks, and will only be required to work directly with your partner to accomplish the integration of those tasks.

“Instructions”

When developing any complicated code/project, it is best to split the effort into small, manageable tasks. The next few subsections of this document will help you build the required functionality for this project one piece at a time. We'll start by transmitting I²C data to the Analog Discovery (AD2). Once we know we can transmit something useful, we'll connect to the RTC and transmit data to the RTC. Finally, we will read data from the RTC.



For each of the “requirement verification boxes”, you need to save screenshots or other data that show a successful verification. You may choose to present this verification data during your presentation if doing so makes sense. The verification data/screenshots should be added to your git repository in the docs folder.

Git procedure

Do all of your I²C bit banging development in an i2c branch. All development until you interact with the RTC should be done in this branch. Once you have verified your I²C code works, submit a pull request and merge your code into the main branch. If you want to, you can use additional branches off the i2c branch and merge those into the i2c branch.

For the RTC development, create and use an rtc branch. Once everything is done, submit a pull request and merge your code into the main branch.

If you have any final documentation to do, you will do that in a new branch of your choosing (e.g., docs).



Your GitHub repositories are set up such that you *can't* push directly to main and pull requests (PRs) must be approved by someone that isn't the PR author (i.e., the other team-mate).



Commit often! At the very least, you should make a commit when you verify that the step is complete.

Heartbeat LED

Create a heartbeat LED using a timer interrupt. You can use either of the LEDs on the dev board. You can use whatever frequency you want (0.5 Hz, 1 Hz, etc.).



Heartbeat LEDs are super common. Engineers like having a visual indicator that the MCU is alive. In this class, you should have a heartbeat LED in every project.

Planning out your I²C functions

Before you start writing any code, read through some I²C app notes and make sure you have a vague understanding of what's going on.

For this project, we want to develop modular code. Doing this will make our code easier to test and to reuse. Think about and document (write down somewhere, either digitally (in your git repo) or on paper) what subroutines you want to create. You will need some delay routines so you can satisfy the timing requirements of the I²C protocol. You will also want subroutines for common I²C tasks like sending start/stop conditions, transmitting/receiving bytes, etc. Here's a list of functions I might use:

- `i2c_init`
- `i2c_start`
- `i2c_stop`
- `i2c_tx_ack`
- `i2c_rx_ack`
- `i2c_tx_byte`
- `i2c_rx_byte`
- `i2c_sda_delay` (for satisfying setup and hold times)
- `i2c_scl_delay` (for setting your clock period)
- `i2c_send_address`
- `i2c_write` (top-level function that would handle an entire write operation)
- `i2c_read` (top-level function that would handle an entire read operation)

You will iteratively develop your code, so don't feel like you need to get your architecture right the first time (you won't). Also note that I'm prone to over-engineering, so keep it simple and do what makes sense to you.



Before implementing any of your functions, create flowcharts for them! You don't need to create flowcharts for everything at the beginning—just create them before you need the function. In future projects, I will force you to create your flowcharts first.

Implementing I²C

Before communicating with an actual I²C peripheral, we will develop and test the I²C code bit-by-bit, using test equipment to verify functionality. We'll set up the AD2's protocol analyzer as an I²C peripheral. This allows us to perform tests in a controlled environment.

Initialize GPIO pins for SDA and SCL

Configure any of the GPIO pins for your SDA and SCL lines, but **do not use any pins reserved for the internal I²C modules**.



Requirement verification

GPIO works (mastery level: C; 2 points)

Once the pins are configured as outputs, verify that you can actually output to them before moving on.

Send a start condition

Write subroutine(s) that produce a start condition. You can assume that SCL and SDA will both be high before calling the start subroutine.



Requirement verification

Start condition (mastery level: C; 4 points)

With a scope or the AD2, verify that you can send a start condition and that it meets the timing requirements specified in the I²C bus-specification.

Send a stop condition

Write subroutine(s) that produce a stop condition.



Requirement verification

Stop condition (mastery level: C; 4 points)

With a scope or the AD2, verify that you can send a stop condition and that it meets the timing requirements specified in the I²C bus-specification.

Transmit an address

Once you can send a start condition, the next step is to send the address (plus the read/write bit).

Since we'll be sending lots of bytes, you will want to develop an `i2c_tx_byte` subroutine (you can choose a different name) that can be used every time you need to send a byte. At this point, you may want to wrap `i2c_start` and `i2c_tx_byte` into an `i2c_write` function that will eventually handle the entire I²C transaction.

Here are some suggested steps for transmitting the address:

1. Create a variable called `tx_byte` (or a better name...). Use that variable to store the address and R/W bit.
2. Implement the `i2c_tx_byte` subroutine:
 - (a) Test the MSB of `tx_byte` to determine what SDA should be, e.g., if the MSB is 1, set SDA.
 - (b) Delay to satisfy the SDA setup time
 - (c) Set SCL
 - (d) Delay for the SCL high time

- (e) Clear SCL
- (f) Delay to satisfy the SDA hold time
- (g) Repeat for the next bit (you may want to shift the bits in tx_byte so you're always testing the MSB, i.e., create a shift register).



Test and step-through your subroutines as you create them. Small changes are easier to debug.



Requirement verification

Address transmission (mastery level: C; 7 points)

Verify that you can send a start condition and address to the AD2. Use the AD2's protocol analyzer to verify that the AD2 interpreted your start condition and address correctly.

Send an acknowledge to the AD2

After sending a byte, the peripheral will send an acknowledge by holding SDA low during the next clock pulse. However, for ease of testing and debugging, it will be easier for us to *send* an acknowledge after sending the address. This way we can send an entire transaction (minus any data packets) and see verify that it is correctly interpreted by the AD2.

Write a subroutine that can send an ACK.



Requirement verification

Full transmission (fake ACK) (mastery level: C; 7 points)

Send a start condition, address (including the r/w bit), ACK, and stop condition to the AD2. Verify that everything is interpreted correctly.

Send multiple bytes to the AD2

Now that we know we can get a full transaction interpreted correctly, try sending multiple data packets after the address. Send an ACK after each data packet.



Requirement verification

Send data packets (fake ACK) (mastery level: C; 7 points)

Verify that you can send multiple data packets in a transmission and they all get interpreted correctly.

Request an acknowledge from the AD2

Now that we've verified we can send full transactions, let's stop faking the acknowledge from the peripheral. Using the AD2's protocol feature, we can set up the AD2 to act as an I²C peripheral that will send ACKs when written to and can send data back to us when we request a read operation. See figure 1 for an example of how the protocol feature can be set up as an I²C peripheral.

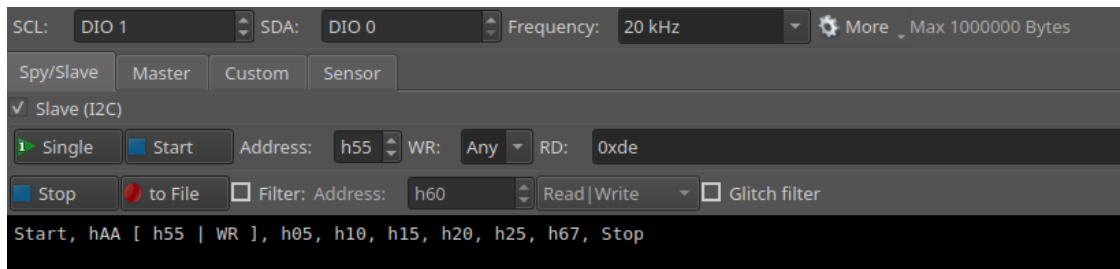


Figure 1: Sample setup of the AD2's protocol analyzer. The I²C transactions will be printed in the console so you can verify what is happening without a logic analyzer.

After sending a byte, the peripheral will send an acknowledge by holding SDA low during the next clock pulse. That means the controller must relinquish control of SDA and set SDA as an input. Read section 3.1.6 of the I²C user manual for more information.

Write a subroutine that can receive an acknowledge or “not acknowledge” (NACK) from the peripheral. When receiving an unexpected NACK, the I²C specification states you can send a stop condition to end the transfer or a repeated start condition to start a new transfer.



When receiving data from the AD2's I²C peripheral, I needed to enable weak internal pull-up resistors on the MSP430's SDA pin. External pull-up resistors will also work.



Requirement verification

Receive ACK from AD2 (mastery level: C; 7 points)

Verify that the AD2 sent an acknowledge and that you received it on SDA. Also verify that the AD2's protocol feature interprets the transaction correctly. You may want to step through your code.

Send dummy data

Now that the AD2 I²C peripheral is responding to us, we can send dummy data to it and simulate an entire write transaction.

Create a count variable that starts at zero and counts up to nine. Send each count to the AD2 as subsequent data packets.



Requirement verification

Simulate write transaction with AD2 (mastery level: C; 10 points)

Verify that your counter data shows up correctly in the AD2's protocol analyzer.



Requirement verification

Generic write subroutine (mastery level: ~~100~~; 2 points)

Write a generic subroutine that can write an *arbitrary* number of bytes to an *arbitrary* peripheral device.

Generic means that the subroutine does not have to be modified in order to change the bytes it's transmitting, how many bytes it's transmitting, and the peripheral's address.

Receive data from the AD2

With the AD2 protocol analyzer in I²C slave mode, we can make the AD2 send data when we send a read request.

You will need to create an `i2c_rx_byte` subroutine. You need to transmit an ACK for each byte you receive from the AD2.



The I²C specification states that the controller must send a NACK to signal the end of a transfer after receiving the last read byte.



Requirement verification

Simulate read transaction with AD2 (mastery level: C; 10 points)

Request and receive one data packet from the AD2. Verify that the byte you save in memory matches what is transmitted by the AD2 (check the protocol console and/or logic analyzer).



Requirement verification

Receive multiple bytes (mastery level: B; 2 points)

Request and receive multiple data packets from the AD2. You don't need to save all the data packets—you can overwrite them if you want.



Requirement verification

Save received bytes (mastery level: A; 4 points)

Save the byte(s) you receive to memory. Show the bytes updating in memory.



Requirement verification

Generic read subroutine (mastery level: 100; 2 points)

Write a generic subroutine that can read and *save to memory* an *arbitrary* number of bytes from an *arbitrary* peripheral device.

Again, *generic* means that the subroutine does not have to be modified in order to change the number of received bytes or the peripheral address.

Communicating with the RTC

Now that you've verified you can perform full I²C write and read transactions, you can confidently move on to interacting with RTC I²C device (which is out of our control, unlike the AD2, which is somewhat under our control). If something doesn't work when interacting with the RTC, we can be fairly certain the problem isn't the basic I²C code you developed—maybe the issue is you didn't hook up the RTC chip correctly, etc., but we can't completely rule out issues with your code...

Wire the circuit

On a breadboard, wire the MCP7940N RTC according to its datasheet and connect it to your MSP430 dev board. The clock crystal and associated capacitors you need are listed in the BOM posted of Canvas under Resources.



Both partners should wire up the RTC on their own breadboard. Going forward, both partners need to build all circuits.

Read the seconds register

Read the seconds register from the RTC. You will need to tell the RTC which register you are wanting to read from first (i.e., send a write transaction with the seconds' register address as the data packet.), then you can send a read transaction to read the seconds register.



Requirement verification

Read RTC seconds (mastery level: B; 3 points)

Continuously read the seconds register and verify that its value changes. You can do this with the AD2's logic analyzer.

Read the seconds, minutes, and hours registers

Continuously read the time in hh:mm:ss—that is, read the hours register first, then minutes, then seconds.



Requirement verification

Read hours, minutes, and seconds (mastery level: B; 3 points)

Verify that the seconds and minutes registers are changing. Save screenshots at two different points in time that show different minutes and seconds values (you don't need to wait an hour for the hour register to update...)



Requirement verification

Save hours, minutes, and seconds (mastery level: A; 4 points)

Save the hours, minutes, and seconds values into variables and show that they are updating.

Going above and beyond 🚀



Requirement verification

Convert BCD to decimal (mastery level: 100; 2 points)

Convert the hours, minutes, and seconds values from BCD into decimal. Show the *decimal* values in memory.



Requirement verification

Set RTC time (mastery level: 100; 2 points)

Set the RTC's time to be close to the actual time.

Demo

Your approximately 5-minute demo/presentation must include:

1. A concise introduction of what the project was
 - Do not assume the instructor or TA knows what you were assigned to do.
 - Focus on the ultimate goal of reading from the RTC.
 - Summarize in approximately 2–3 sentences.
2. A presentation of your circuit diagram. Please explain.
3. A discussion of how you developed the I²C bit banging code.
 - Provide screenshots showing successful I²C transactions with the AD2.
4. A presentation of your top-level flowcharts.
 - One flowchart should show the overall program flow of interacting with the RTC.
 - Show high-level flowcharts for a few of the most important subroutines (e.g., your generic I²C write routine).
5. The actual demo. Demonstrate your functioning code. Demonstrate transactions with the RTC.